Untypedscript

Group 36
Erwin Hörnell, Felix Svensson, Alexander Thoresson

This is our project for PKD 2026

March 10, 2026

Contents

1 Introduction **3**

2 Use Cases **3**

3 Program documentation **4**

3.1 Overview 4
3.2 Scanner 4
3.3 Parser 10
3.3.1 Context-Free Grammar 10
3.3.2 Recursive Descent Parsing 12
3.4 Interpreter 14
3.5 Main file and error handling 16
3.6 Tests 18

4 Discussion **20**

5 Conclusion **20**

1 Introduction

The goal of this project was to apply as much of the knowledge acquired during the course as possible in a single system. To achieve this, we chose to design and implement a small programming language together with an interpreter capable of executing programs written in that language. The resulting language, called *UntypedScript*, is interpreted by a custom interpreter implemented in TypeScript. Apart from the standard data types introduced during the course, no external libraries were used.

The design of UntypedScript was intentionally kept flexible during development. The initial goal was to create a dynamically typed language in which most constructs evaluate to values, allowing expressions to be used in a wide range of contexts. We also aimed to support labelled loops if time permitted. Beyond these initial ideas, the language evolved incrementally as the interpreter was implemented.

In its current form, UntypedScript supports arithmetic and logical expressions, variable declarations, block structures with scoped variables, user-defined functions, function overloading, labelled `while` loops, and printing values to the terminal. Although the language does not provide the full feature set of modern programming languages, the project provided valuable insight into the design and implementation of interpreters, parsing techniques, environment models, and collaborative development using version control.

2 Use Cases

The primary purpose of the interpreter is to execute programs written in UntypedScript from a source file. While the language itself is intentionally minimal, it does serve as a platform for experimenting with language design and interpreter implementation.

Because UntypedScript currently lacks built-in libraries and input functionality, it is primarily suited for small demonstration programs and experiments with control flow, expressions, and functions. Typical use cases include evaluating arithmetic expressions, testing language constructs such as labelled loops, and exploring how different syntactic structures are interpreted.

3 Program documentation

3.1 Overview

The interpreter is divided into three main modules: `scanner`, `parser`, and `interpreter`. Each module has a distinct responsibility, which improves modularity and makes the overall system easier to understand and maintain.

The `scanner` reads the source program as a sequence of characters and converts it into a sequence of tokens. These tokens represent meaningful elements of the language, such as identifiers, keywords, operators, and literals.

The `parser` takes the sequence of tokens as input and constructs a set of Abstract Syntax Trees (ASTs). Each AST represents the hierarchical structure of a statement or expression according to the language grammar.

Finally, the `interpreter` evaluates the generated ASTs in sequence. During this stage, the interpreter executes the program by traversing the AST nodes and performing the operations they represent.

The program also contains minor modules for reading source code from a file and error reporting.

3.2 Scanner

A string of text can only really be worked with one character at a time, and a source file consists of various different meaningful units that vary from a single character up to an unlimited length. It also has text that has no meaning to the execution of the program at all such as line breaks, whitespace and comments. Having a separate component first identify and convert these units into a uniform data structure helps simplify the latter, more complex step of building the AST. The parser can then assume this token structure as the smallest meaningful unit without having to deal with any string manipulation. In this stage we can also catch the most obvious syntax errors, like unrecognized characters and malformed number literals, and halt execution before doing any more unnecessary work.

Token structure

```
type Token = {
    type: TokenType,
    index: number,
    value: number | string | undefined,
}
```

The tokens store three types of information: The type of token, the place in the source code it was parsed and an optional value. All tokens have a type,

and for most of them this is all that's required for the parsing to work. We store these as an enum value, since all we need to do with these is decide how we're going to handle the token and having a clearly defined set of possible values helps minimize the risk of bugs introduced as the complexity of the parser and interpreter grows. However, we also want the debugging part of using Untypedscript to be as user friendly as possible. By also storing the position of the token in the source code, we can later on use this information to show exactly where in the code an error occurred.

Finally, the language supports three types of tokens which carry an additional value: Strings, numbers and identifiers. Strings and numbers are among the basic data types, and identifiers represent variable or function names. Since their values are simply written out in the source code we can parse these at this stage and pass them on to the parser as single units.

Example of token types

```
enum TokenType = {
    COMMA, PLUS, MINUS, RIGHT_PAREN, LEFT_PAREN ...
    NUMBER_LIT, STRING_LIT, IDENTIFIER
}
```

The interface to the scanner has the signature:

```
function scan(input: string):
    Array<Token> | Array<UntypedscriptError>
```

where the input is a string of source code, and the result depends on if any syntax errors were encountered. If the scanning was successful, the sequence of Tokens is returned as an array, while if any errors occured these will all be returned. We could have used a list structure here, but chose to use arrays instead to cut down on external libraries. Since we only ever read them by looping over the whole thing, we don't need to worry about potentially reading undefined values. However, in the end we still ended up needing the list library since the hashtable library depend on it. We also could have implemented this by throwing an error as soon as invalid syntax was encountered, but by finishing the scanning then returning all the errors at once we can then show them all to the user at once, allowing them to easily fix them in one go rather than having to rerun the program one fix at a time.

The scanner works by initializing with an index of 0. We then look at the character at that index, and pass this into a big switch statement.

```
    case ",":
        output.push(make_token(TokenType.COMMA));
        break;
```

```
    case "*":
        if (peek(1) === "*") {
            output.push(make_token(TokenType.POW));
            advance();
        } else {
            output.push(make_token(TokenType.TIMES));
        }
        break;
```

For simple single character tokens, we can simply output the corresponding token and move on. For tokens that could possibly consist of multiple characters, such as = and ==, we also look one character ahead and output the correct token based on what follows. If we match a quotation mark, we try to scan a string literal which simply ends at the next quotation mark. We could have made separate syntax for multi line strings, but just allowing simple string literals to span multiple lines was the simplest solution both to implement as well as to use.

Finally, we end up in the default case:

```
default:
    if (is_whitespace(ch)) {
        break;
    } else if (is_digit(ch)) {
        const n = scan_number();
        if (n !== null) {
            output.push(n);
        }
        continue;
    } else if (is_letter(ch) || ch == "_") {
        const ident = scan_identifier();
        const keyword = ph_lookup(keywords, ident.value
           as string);
        output.push(
            keyword !== undefined
                ? token(ident.index, keyword)
                : ident
        );
        continue;
    }else {
        error("Unrecognized character '" + ch + "'");
        skip_line = true;
    }
    break;
```

Whitespace characters are simply ignored, this includes line breaks, tabs and spaces. If the character is a digit we try to scan a number literal and if it is a letter or underscore we try to scan an identifier. If we still can't match it, we output a syntax error and skip the rest of the line. The reason we skip the line is because if we encounter a syntax error the rest of the line is likely to be invalid as well, and by skipping to the next line we make sure not to spam the user with meaningless errors. This is done simply by setting a flag that, if set, at the beginning of the loop will simply keep advancing the scanner to the next character until we either hit the end of file or a newline character.

After the switch statement we advance to the next character and loop back around. If we hit the end of file we check if there were any errors and return these or the successful tokens accordingly:

```
case "\0":
    if (!scanner.errored) {
        output.push(make_token(TokenType.EOF));
        return output;
    } else {
        return errors;
    }
```

For parsing literals and identifiers we use a few helper methods: scan_string, scan_number and scan_identifier. scan_string isn't very interesting, it just keeps advancing until it hits another quotation mark or end of file, then returns a substring of the source code between these. Scan_number however is a bit more involved:

A number literal in untypescript can be either an integer or a decimal number, demarcated by a '.'. We know we started at a digit, so we keep advancing as long as we keep getting more digits. If we get something else, we check if it's a dot. If that's the case, we look at the character after the dot to make sure it's a digit. This is to prevent things like '3.a'. In many object-oriented languages this could be valid syntax for a method call of 'a' on the number object 3, but since the current iteration of untypescript doesn't have classes or support for this method call syntax this isn't something that we need to handle.

If the character after the dot is a digit, we simply keep advancing until we hit something else. We then use the built in Number parsing constructor to parse the substring between start and end index into a number.

The identifier scanning is a bit simpler, since we just advance while the character is one of the ones that we support in our identifier names (letters, numbers and underscores). This is also how we handle keywords like if and

Algorithm 1: scan_number

Result: A number literal
start ← scanner index;
while *character is digit* **do**
  advance();
if *next character is '.'* **then**
  if *next character is not digit* **then**
    error(...);
    skip_line ← *true*
    return *null*
while *character is digit* **do**
  advance();
end ← scanner index;
$n \leftarrow Number(substring(source, start, end))$;
return *number literal token n*

else: after scanning an identifier we do a hash table lookup against a table of keywords. If there's a match, we output the corresponding token instead. We could have handled this separately in the switch statement, like so:

```
    case "f":
        // Look ahead one character
        switch(peek(1)) {
            case "n":
                // fn keyword
            case: "a"
                // keep checking for false
        }
```

However, not only would this end up with a very messy switch statement, it would also not let the user use identifiers that start with a keyword. For instance, 'var' is a keyword in untypescript for function declarations, but this would also disallow identifiers like 'variable' from being used. You could do even more checks inside the different cases to fix this, but the simpler approach is simply to scan as long of an identifier as we can, and then check if it matches one of the keywords.

We do this by storing all the possible keywords together with their corresponding Token in a hashtable. We do this since it gives us an easy way to look up a string and get back a corresponding token. As the hash function we simply take the character code of the first letter, both because almost all

of our keywords end up starting with different letters but also because it's very simple (since the hashtable is so small the performance implications of one or two conflicts is irrelevant).

3.3 Parser

The parser takes an array of tokens as input and returns either an array of ASTs (Abstract Syntax Trees) or an array of errors if any are encountered. If the parser detects missing or unexpected tokens, it records an error and attempts to recover before continuing the parsing process. This allows multiple errors to be reported from a single run, improving the user experience by providing more comprehensive feedback.

Individual tokens cannot be interpreted in isolation, as their meaning often depends on their position relative to other tokens. Instead, tokens are grouped into larger syntactic components such as statements and expressions. These components can recursively contain other components, forming a hierarchical tree structure that represents the program. Each node in this structure has a specific type, a set of fields, and rules that define what child nodes it may contain. These rules are described using a context-free grammar (CFG).

3.3.1 Context-Free Grammar

$\langle \textit{Program} \rangle$ ::= $\langle \textit{Statement} \rangle$*

$\langle \textit{Statement} \rangle$ ::= $\langle \textit{VarDecl} \rangle$
| $\langle \textit{FunctionDecl} \rangle$
| $\langle \textit{ReturnStmt} \rangle$
| $\langle \textit{PrintStmt} \rangle$
| $\langle \textit{BreakStmt} \rangle$
| $\langle \textit{ExpressionStmt} \rangle$

$\langle \textit{VarDecl} \rangle$ ::= `var` IDENTIFIER (`=` $\langle \textit{Expression} \rangle$)? `;`

$\langle \textit{FunctionDecl} \rangle$ ::= `fn` IDENTIFIER `(` $\langle \textit{Parameters} \rangle$? `)` $\langle \textit{Block} \rangle$

$\langle \textit{Parameters} \rangle$ ::= IDENTIFIER (`,` IDENTIFIER)*

$\langle \textit{ReturnStmt} \rangle$ ::= `return` $\langle \textit{Expression} \rangle$ `;`

$\langle \textit{PrintStmt} \rangle$ ::= `print` $\langle \textit{Expression} \rangle$ `;`

$\langle \textit{ExpressionStmt} \rangle$ ::= $\langle \textit{Expression} \rangle$ `;`

$\langle \textit{Expression} \rangle$ ::= $\langle \textit{WhileExpr} \rangle$
| $\langle \textit{LoopExpr} \rangle$
| $\langle \textit{IfExpr} \rangle$
| $\langle \textit{Assignment} \rangle$

$\langle \textit{WhileExpr} \rangle$ ::= `while` $\langle \textit{Expression} \rangle$ (`:` IDENTIFIER)? $\langle \textit{Block} \rangle$

$\langle \textit{LoopExpr} \rangle$ ::= `loop` (`:` IDENTIFIER)? $\langle \textit{Block} \rangle$

$\langle \textit{IfExpr} \rangle$ ::= `if` $\langle \textit{Expression} \rangle$ $\langle \textit{Expression} \rangle$ (`else` $\langle \textit{Expression} \rangle$)?

$\langle \textit{BreakStmt} \rangle$ ::= `break ;`
| `break :` IDENTIFIER `;`
| `break return` $\langle \textit{Expression} \rangle$ `;`

$\langle \textit{Assignment} \rangle$ ::= IDENTIFIER `=` $\langle \textit{Expression} \rangle$ `;`
| $\langle \textit{LogicOr} \rangle$

$\langle \textit{LogicOr} \rangle$ ::= $\langle \textit{LogicAnd} \rangle$ (`or` $\langle \textit{LogicAnd} \rangle$)*

$\langle \textit{LogicAnd} \rangle$ ::= $\langle \textit{Equality} \rangle$ (`and` $\langle \textit{Equality} \rangle$)*

$\langle \textit{Equality} \rangle$ ::= $\langle \textit{Comparison} \rangle$ ((`==` | `!=`) $\langle \textit{Comparison} \rangle$)*

$\langle \textit{Comparison} \rangle$ ::= $\langle \textit{Term} \rangle$ ((`<` | `<=` | `>` | `>=`) $\langle \textit{Term} \rangle$)*

$\langle \textit{Term} \rangle$ ::= $\langle \textit{Factor} \rangle$ ((`+` | `-`) $\langle \textit{Factor} \rangle$)*

$\langle \textit{Factor} \rangle$ ::= $\langle \textit{Exponent} \rangle$ ((`*` | `/`) $\langle \textit{Exponent} \rangle$)*

$\langle \textit{Exponent} \rangle$ ::= $\langle \textit{Unary} \rangle$ (`^` $\langle \textit{Unary} \rangle$)*

$\langle \textit{Unary} \rangle$ ::= (`-` | `!`) $\langle \textit{Unary} \rangle$
| $\langle \textit{Call} \rangle$

$\langle \textit{Call} \rangle$ ::= $\langle \textit{Primary} \rangle$ (`(` $\langle \textit{Arguments} \rangle$? `)`)*

$\langle Arguments \rangle$ ::= $\langle Expression \rangle$ (`,` $\langle Expression \rangle$)*

$\langle Primary \rangle$ ::= NUMBER
| STRING
| `true`
| `false`
| `null`
| IDENTIFIER
| `(` $\langle Expression \rangle$ `)`

$\langle Block \rangle$ ::= `{` $\langle Statement \rangle$* `}`

The terminals of this grammar consist of keywords, literals, identifiers, braces, parentheses, commas, colons, semicolons, and operators. The grammar was partially designed before implementation began and served as a guide during development. As the parser was implemented, additional rules were refined and incorporated into the grammar.

An important architectural feature of the grammar is the hierarchical structure of expressions. Higher-level expressions produce either a single lower-precedence expressions followed by repeated operator–expression patterns such as `( )*`. This structure naturally enforces operator precedence and associativity in the language. Unary operations have their operands before the value, so we handle these a little differently through recursion to make things a little cleaner.

3.3.2 Recursive Descent Parsing

Recursive descent parsing was used to implement the grammar and enforce the correct order of operations in expressions. In this approach, each nonterminal in the grammar corresponds to a parsing function. These functions call one another recursively, mirroring the hierarchical structure defined by the grammar.

A clear example of this design can be seen in how the parser handles *terms* and *factors*. The functions `parse_term()` and `parse_factor()` implement two adjacent levels of operator precedence.

Algorithm 2: parse_term

```
Result: An expression AST node
term ← parse_factor();
while next token is + or − do
    index ← previous token index;
    operator ← sign(previous token);
    right ← parse_term();
    return make_binary(operator, term, right, index);
return term;
```

Algorithm 3: parse_factor

```
Result: An expression AST node
fact ← parse_exponent();
while next token is *, or / do
    index ← previous token index;
    operator ← sign(previous token);
    right ← parse_factor();
    return make_binary(operator, fact, right, index);
return fact;
```

The ASTs produced by `parse_term()` may contain either a factor or another term as a direct child node. In contrast, a factor node cannot contain a term as a direct child. This structure ensures that multiplication and division occur before addition and subtraction.

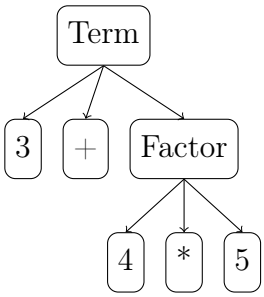

Figure 1: Parse tree for the expression $3 + 4 * 5$

3.4 Interpreter

The interpreter receives the AST created with the parser, and interprets all of the expressions and/or statements in the array with the interpret function. The final value is returned after going through the entire AST. Using a switch statement, interpret has different cases for every type of "Statement". Depending on the type, either the expression of the statement is evaluated, or the statement is declared. The types for the cases are "Return", "Print", "Expression_statement", "Variable_declaration", "Break", and "Function_declaration". The default is when expr is an "Expression", which returns the evaluated expression.

Shortened example of interpret

```
export function interpret(expr: Expression | Statement):
    Value {
     switch (expr.type) {
         ...
         ...
         case "Expression_statement":
             evaluate(expr.expression);
             return null;
         ...
         ...
         case "Function_declaration":
             declare(expr as FunctionDec);
             return null;
         default:
             return evaluate(expr);
     }
}
```

The evaluate function evaluates a given expression with a different helper function depending on the type of the expression. An expression can have the types "Literal", "Unary", "Binary", "Logic", "Block", "While", "Variable", "If", "Call", and "Assignment".

Shortened example of evaluate

```
function evaluate(expr: Expression): Value {
    switch (expr.type) {
        case "Literal":
            return literal(expr);
        ...
        ...
        ...
        case "Block":
            return block(expr);
        ...
        ...
        ...
        ...
        ...
    }
}
```

Literal is a simple function that returns the value of the "Literal" expression, but functions for the other types became more complex. An example

is a block, which is anything between curly braces, represented in our code as an array of "Expression" and/or "Statement". We use the block as the body of our functions and loops.

The block function

```
function block(block: Block): Value {
    let return_value: Value = null;
    enter_frame(null);
```

Blocks uses frames, which is a stack of "Frame", where each "Frame" is a probing hashtable. We use frames to make scoped variables work. The function enter_frame() pushes an empty "Frame" onto the frame stack.

```
    for (let i = 0; i < block.body.length; i += 1) {
        return_value = interpret(block.body[i]);
```

We call each expression in the block with interpret.

```
        if (should_break === true || typeof should_break
           === "string") {
            if (should_break === block.label) {
                should_break = false;
            }
            break;
        }
    }
```

This is a check to see if the block should end early.

```
    pop_frame();
    return return_value;
}
```

Pops the top "Frame" off the frame stack, and returns the value of the block after all the expressions have been interpreted.

3.5 Main file and error handling

The final components of the program are the main file, interpret_source and error. The main file is what you run to read in and interpret a source file, and is very bare bones. It only contains what's required to get the user specified file name, resolve that to an absolute path and read the contents. It then calls interpret_source with the resulting text string. The reason we split the file reading and text interpreting up into different functions is to enable us to write tests for the main file that don't involve file reading. Requiring a

separate test file for each test would quickly get very messy, so by allowing us to simply pass in a text string made this testing much easier.

The interpret_source function is what ties the three components together:

```
export function interpret_source(source: string): Value |
    Array<UntypedscriptError> {
    const res = scan(source);
    if (has_errors(res)){
        return res;
    }

    const parsed = parse_tokens(res as Array<Token>);
    if (has_errors(parsed)){
        return parsed;
    }

    try {
        return interpret_results(parsed);
    } catch (e) {
        const error = e as UntypedscriptError;
        return [error];
    }
}
```

It takes a string of source code, then passes this to the scanner. In each step of the process, it checks to see if any errors occurred, then returns these to the main file where they're displayed. If successful, it passes the resulting sequence of tokens onto the parser, which parses them into an AST. Finally, this is passed on to the interpreter. This is wrapped in a try/catch block, since unlike the scanner and parser syntax errors we handle runtime errors by throwing these. We do this since it doesn't really make sense to try to recover from these, so we want to exit the program as soon as we encounter one. We still pass these errors on to the main file for display to the user, however.

The error handling is done in the main file, by first initializing the error handler. This is done with the following function:

```
function init(source: string): (es: Array<
    UntypedscriptError>) => void
```

It takes the same string of source code that is passed to the interpret_-source function, and stores this state internally. It then returns a function that can be used to display an array of errors. It was written this way early on in the project to avoid using global variables, though we ended up

using these in the interpreter later on anyway. Internally, this simply calls display_errors, which iterates through the array and calls display_error on each error. This function in turn is simply a lot of string manipulation code to look at the error's index and get the corresponding line (and the adjacent lines) of the source file. It also creates a pointer that underlines exactly where the error occured, to make it very clear to the user what actually went wrong. Finally it prints all of this to the console.

Example of an error display

```
Error: Break label 'label' doesn't exist
1 |         var x = true;
2 |         break: label;
  |                ^^^^^ Here
3 |         var y = false;
```

3.6 Tests

We wrote tests for the scanner, parser, and main file. Before main was functional, we needed to test the scanner and later the parser to see if they functioned like we wanted.

For the scanner, the tests compare the result of using the scan function with different strings, and a manually created array of tokens to see if they are strict equal.

Example of a scanner test

```
test("Scan a single digit", () => {
        const s = "1";
        const expected = [
            token(0, TokenType.NUMBER_LIT, 1),
            token(1, TokenType.EOF)
        ];
        expect(scan(s)).toStrictEqual(expected);
    })
```

The parser tests include a lot of different comparisons, but there are three more common ones. The first one, is if the result of parsing the manually made array of tokens resulted in any errors. The second is the expected length of the output of the result from the parsing. The third is the expected type for the first output of the result from the parsing.

Example of a parser test

```
test("parses number literal", () => {
        const tokens = [
            token(TokenType.NUMBER_LIT, 42, 0),
            token(TokenType.SEMICOLON, null, 2),
            token(TokenType.EOF, null, 3),
        ];

        const result = parse(tokens);

        expect(result.has_error).toBe(false);
        expect(result.output.length).toBe(1);
        expect(result.output[0].type).toBe("
           Expression_statement");
    });
```

Those tests made sure that the functions worked and could be called upon. After the interpreter was implemented, we also created tests for the main file running the program. The tests are grouped under these titles, "Interpret literals", "Interpret binary operators", "Associativity and orders of operation", "Interpret unary operators", "Declaration and assignment", "Loops", "Function calls", "Logical operators", "If/else", "Errors". We made the different groups of tests to try to encompass all of the components of our language. The following is a test under the group "Function calls".

Example of a main test

```
test("Multiple arguments", () => {
        const s = "fn add(a, b, c) { a + b + c } add(1,
           2, 3)";
        expect(interpret_source(s)).toBe(6);
    });
```

This test is to see if the function interpret_source, (interprets a string written in our language and returns either a value or an array of UntypedscriptError) can take a string written in our language, with multiple arguments, and return the expected value.

4 Discussion

Some aspects of the design and implementation may be suboptimal. Firstly, the division of AST node types into *Expressions* and *Statements* is somewhat unclear, and the distinction between the two changed several times during development. This was partly caused by the intended syntax of UntypedScript, where most constructs return a value. At the same time, we attempted to adhere to conventions from the C programming language, where statements and expressions are more clearly separated. These competing design goals made the overall structure of the AST less consistent than initially intended.

Secondly, the way scoping is currently handled in the interpreter makes it difficult to implement lexical scoping for functions, as well as returning functions and assigning them to variables. Supporting such features would make UntypedScript a more flexible and expressive language. However, implementing a more robust environment model would have required significantly more development time. Given the limited time available, the current implementation allowed us to focus on delivering core functionality.

As a consequence, the present design of the interpreter may become a bottleneck for further development. Nevertheless, it enabled the implementation of a working language with several key features within the time of the project.

5 Conclusion

In this project, we designed and implemented a small programming language together with an interpreter capable of executing programs written in that language. Due to the ambitious scope of the project and the limited time available, some design and architectural decisions were made that may not

be optimal. Nevertheless, the final result is a working interpreter capable of parsing and executing UntypedScript programs while also reporting errors encountered during scanning, parsing, and interpretation.

Developing the interpreter required the application of many concepts introduced throughout this course, as well as the Automata Theory course, including environment-based evaluation, abstract syntax trees and context-free grammars. Implementing these components helped deepen our understanding of how programming languages are structured and how source code can be transformed into executable behaviour.

Although UntypedScript is intentionally minimal and lacks many features found in modern programming languages, the project demonstrates the core mechanisms required to implement a language interpreter. The experience also provided valuable insight into software design, debugging, and collaborative development using version control.